

Disciplina: Teoria dos Grafos

PROJETO FINAL — TEORIA DOS GRAFOS

Rede de Aeroportos Brasileiros e Rede de Filmes IMDb

Relatório Técnico Final

Equipe:

Guilherme Alencar Augusto Correa — gaac@cesar.school

Rodrigo Lucena Cavalcanti — rlc2@cesar.school

Rodrigo Torres Galindo Filho — rtgf@cesar.school

Erick Acioli Belo — eab2@cesar.school

Professor(a): Laura Pacífico

Data: 11/06/2026

1. Resumo

Este relatório apresenta o Projeto Final da disciplina de Teoria dos Grafos, desenvolvido com o objetivo de modelar dois problemas reais como grafos e aplicar algoritmos clássicos de exploração, conectividade e caminhos mínimos. O projeto foi dividido em duas partes: a primeira voltada à representação da malha aérea brasileira e a segunda voltada à construção de uma rede de filmes baseada em dados derivados do IMDb.

A Parte 1 modela uma rede de aeroportos brasileiros como um grafo não direcionado e ponderado. Cada aeroporto é representado por um vértice identificado pelo código IATA, e cada rota aérea direta é representada por uma aresta ponderada pelo tempo estimado de voo. O grafo possui 20 vértices, 114 arestas e densidade 0,60, indicando alta conectividade em relação ao número máximo de conexões possíveis. A modelagem permite analisar hubs nacionais, conectividade regional e caminhos mínimos entre aeroportos.

A Parte 2 modela uma rede de filmes como um grafo não direcionado e ponderado. Cada filme é representado como vértice, e dois filmes são conectados quando compartilham atores e/ou gêneros. A similaridade entre filmes é convertida em peso por meio de uma relação inversa, permitindo o uso do algoritmo de Dijkstra para encontrar caminhos de maior proximidade semântica. O grafo final possui 3.899 vértices, 63.484 arestas e uma única

componente conexa, demonstrando que todos os filmes presentes no dataset podem ser alcançados por alguma sequência de relações.

Os algoritmos BFS, DFS, Dijkstra e Bellman-Ford foram implementados manualmente em Python puro, sem bibliotecas externas de grafos como NetworkX ou igraph. As implementações foram validadas por 13 testes automatizados com pytest. Além dos algoritmos, o projeto inclui geração de métricas, visualizações estáticas, visualizações interativas, benchmark de desempenho e uma interface web funcional para exploração da rede de filmes.

2. Introdução

Neste projeto, a Teoria dos Grafos foi aplicada a dois domínios distintos. O primeiro domínio é a malha aérea brasileira, em que aeroportos e voos diretos formam naturalmente uma rede de conexões. O segundo domínio é uma rede cultural de filmes, em que obras cinematográficas podem ser aproximadas por características compartilhadas, como atores e gêneros.

O primeiro tema abordado, chamado de Parte 1, foi desenvolvido pela professora Laura Pacifico, na qual ela estabelece qual aeroporto deve ser considerado ou não, por mais que na rede de aeroportos do Brasil existam muitos aeroportos, fomos encaminhaados a fazer com os selecionados. Na rede de aeroportos, o foco está em conectividade física, hubs e caminhos de menor tempo. Na rede de filmes, o foco está em similaridade, recomendação e navegação entre conteúdos relacionados. Essa diversidade permitiu testar os algoritmos em grafos com características bem diferentes: um grafo pequeno e denso na Parte 1 e um grafo maior e mais esparsos na Parte 2.

3. Objetivos do Projeto

3.1 Objetivo Geral

Modelar dois problemas reais como grafos, implementar algoritmos fundamentais de Teoria dos Grafos em Python puro, analisar as propriedades estruturais das redes resultantes e produzir visualizações e documentação técnica adequadas a uma disciplina universitária de Ciência da Computação.

3.2 Objetivos Específicos

- Representar a malha aérea brasileira como um grafo não direcionado e ponderado, em que vértices são aeroportos e arestas são rotas diretas.
- Representar uma rede de filmes derivados do IMDb como um grafo não direcionado e ponderado, em que vértices são filmes e arestas indicam similaridade.
- Definir pesos coerentes com o domínio de cada problema: tempo de voo na rede aérea e custo inversamente proporcional à similaridade na rede de filmes.

- Implementar manualmente os algoritmos BFS, DFS, Dijkstra e Bellman-Ford.
- Calcular métricas estruturais como ordem, tamanho, densidade, grau médio, hubs e componentes conexas.
- Gerar visualizações estáticas e interativas para apoiar a interpretação dos resultados.
- Construir uma interface web para exploração da rede de filmes, incluindo busca, visualização e roteamento entre pares de filmes.
- Validar os algoritmos por meio de testes automatizados.
- Documentar decisões técnicas, limitações e possíveis melhorias futuras.

4. Metodologia Geral

4.1 Abordagem de Modelagem

A metodologia adotada partiu do processo clássico de modelagem de grafos. Primeiro, foram identificadas as entidades relevantes de cada domínio. Em seguida, foram definidas as relações entre essas entidades. Depois, cada relação recebeu um peso com significado técnico adequado ao problema. Por fim, os grafos foram classificados quanto à direção, presença de pesos e estrutura geral.

Na Parte 1, a entidade principal é o aeroporto, e a relação entre aeroportos é a existência de voo direto. Na Parte 2, a entidade principal é o filme, e a relação entre filmes é a presença de atributos compartilhados. Apesar das diferenças de domínio, ambas as partes seguem a mesma lógica computacional: construir uma lista de adjacência ponderada e aplicar algoritmos de exploração e caminhos mínimos.

4.2 Fonte e Tratamento dos Dados

Na Parte 1, os dados foram organizados em arquivos CSV compartilhados pela professora Laura Pacifico, contendo aeroportos, conexões entre aeroportos e pares de rotas a serem analisadas. Os aeroportos foram identificados por códigos IATA, acompanhados de informações auxiliares como cidade e região. As rotas foram consideradas apenas quando havia evidência de voo direto, evitando representar conexões com escalas como se fossem arestas diretas.

Na Parte 2, o dataset foi escolhido por meio do Kaggle, consolidado em dois arquivos principais: um arquivo de vértices, contendo os filmes e seus atributos, e um arquivo de arestas, contendo os pares de filmes conectados, os atributos compartilhados, a similaridade e o peso. A separação entre vértices e arestas segue diretamente a formulação formal de grafos e facilita tanto o carregamento quanto a validação dos dados.

O tratamento dos dados incluiu verificação de duplicatas, padronização de identificadores e validação de consistência estrutural. Na rede de filmes, a fórmula de similaridade foi construída com base em atores/atrizes e gêneros compartilhados. A informação de diretor foi removida da fórmula final após a constatação de que introduziu conexões pouco

representativas em alguns casos, como filmes do mesmo diretor, mas com pouca relação temática ou de elenco.

4.3 Ferramentas Utilizadas

O projeto foi desenvolvido principalmente em Python puro. Os algoritmos de grafos foram implementados manualmente, sem uso de bibliotecas prontas como NetworkX ou igraph. Essa decisão foi importante para atender ao objetivo acadêmico da disciplina, pois exigiu a implementação direta das estruturas e das lógicas algorítmicas.

Foram utilizados arquivos CSV como entrada de dados, JSON como saída estruturada para relatórios e interface, HTML/CSS/JavaScript para a interface web, vis-network local para visualização interativa, matplotlib para visualizações estáticas e pytest para testes automatizados.

4.4 Estrutura Geral do Projeto

Componente	Função	Arquivos Relacionados	Resultado Produzido
src/	Módulo principal de código Python	solve.py, cli.py, viz.py	Pipeline de execução
src/parte2/	Módulos específicos da Parte 2	build_interface_data.py, build_interface_grafo.py	JSONs da interface
data/	Dados de entrada da Parte 1	aeroportos_data.csv, adjacencias_aeroportos.csv, rotas.csv	Grafo da Parte 1
data/dataset_parte2/	Dados de entrada da Parte 2	lmdb_filmes.csv, lmdb_arestas.csv, artificiais_bellman_ford/	Grafo IMDb e grafos artificiais
out/	Saídas geradas pelos scripts Python	.json, .csv, .html, .png	Métricas, visualizações e relatórios
interface/	Interface web da Parte 2	app.js, index.html, data/.json, assets/.png	Aplicação navegável
tests/	Testes automatizados	test_*.py	13 testes validados

4.5 Pipeline de Execução

O pipeline segue um fluxo reprodutível: leitura dos CSVs, construção dos grafos, execução dos algoritmos, geração de métricas, criação de arquivos de saída e disponibilização dos resultados na interface.

Etapa	Comando	Saída
Execução completa	<code>python -m src.solve</code>	Todas as saídas de ambas as partes
Parte 1 isolada	<code>python -m src.solve parte1</code>	Métricas e visualizações da rede aérea
Parte 2 isolada	<code>python -m src.solve parte2</code>	Relatório JSON e benchmark da rede IMDb
Visualizações Parte 2	<code>python -m src.viz parte2</code>	PNGs em <code>out/parte2/</code> e <code>interface/assets/</code>
Dados da interface	<code>python -m src.parte2.build_interface_data</code>	<code>resumo_parte2.json</code> e <code>parte2_amostra.json</code>
Grafo completo JSON	<code>python -m src.parte2.build_interface_grafo</code>	<code>parte2_grafo.json</code>
Servidor web local	<code>python -m http.server 8000 --bind 127.0.0.1</code>	Interface em <code>http://127.0.0.1:8000/interface/</code>
Testes automatizados	<code>python -m pytest -v</code>	13 testes aprovados

5. Parte 1 — Rede de Aeroportos Brasileiros

5.1 Objetivo da Parte 1

O objetivo da Parte 1 foi modelar uma rede de aeroportos brasileiros como grafo, permitindo analisar conectividade, centralidade de hubs, distribuição regional de conexões e caminhos mínimos entre pares de aeroportos. Essa modelagem representa um problema clássico de redes de transporte, em que vértices são pontos físicos e arestas são ligações diretas.

5.2 Modelagem do Grafo

5.2.1 Vértices

Cada vértice representa um aeroporto brasileiro presente no dataset. O identificador adotado foi o código IATA, como REC para Recife, GRU para Guarulhos, BSB para Brasília e MAO para Manaus. Essa escolha é adequada porque o código IATA é curto, padronizado e único dentro do domínio aeroportuário, facilitando a indexação e a visualização.

Além do identificador, cada vértice possui atributos auxiliares, como cidade e região. Esses atributos não alteram a topologia do grafo, mas permitem análises adicionais, como comparação de conectividade por região e identificação de padrões geográficos.

5.2.2 Arestas

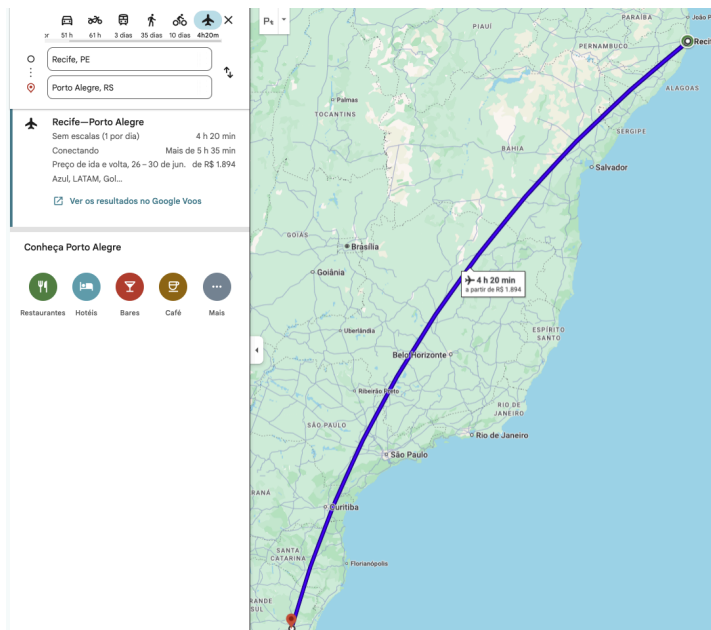
Cada aresta representa uma rota aérea direta entre dois aeroportos. O grafo foi modelado como não direcionado, pois a existência de uma rota direta entre dois aeroportos foi tratada como uma conexão operacional bidirecional. Assim, uma rota REC ↔ GRU representa a possibilidade de conexão entre Recife e Guarulhos nos dois sentidos.

A decisão de usar um grafo não direcionado simplifica a análise e é coerente com o objetivo do projeto: estudar conectividade estrutural da malha aérea, e não variações comerciais específicas de frequência, preço ou disponibilidade por sentido.

5.2.3 Pesos

O peso de cada aresta representa o tempo estimado de voo direto entre os aeroportos exclusivamente analisados por meio do Google Maps (modo avião), medido em horas e transformado em número decimal usando a lógica de proporcionalidade. Essa escolha é tecnicamente adequada porque transforma o problema de caminho mínimo em uma busca pela rota com menor tempo total estimado.

Pesos menores indicam conexões mais rápidas. Pesos maiores indicam trajetos mais longos. Como os tempos de voo são sempre positivos, o grafo satisfaz a pré-condição do Dijkstra, que exige pesos não negativos.



5.2.4 Hierarquia de Aeroportos

Para controlar a plausibilidade das conexões e organizar a estrutura da rede, os aeroportos foram classificados em cinco níveis hierárquicos:

- Nível 1 — Super Hubs Nacionais: GRU, BSB e GIG. São aeroportos de maior conectividade, responsáveis por articular a rede nacional.

- Nível 2 — Hubs Regionais: CNF, REC, SSA e FOR. Atuam como centros regionais relevantes, conectando regiões específicas aos super hubs.
- Nível 3 — Intermediários: CWB, POA, FLN, VIX, GYN, MAO, BEL e CGH. Possuem conectividade moderada, com importância regional ou estratégica.
- Nível 4 — Regionais: NAT, JPA e THE. Apresentam conectividade mais limitada, normalmente dependente de hubs.
- Nível 5 — Periféricos: PVH e RBR. São aeroportos com menor conectividade, integrados à rede principalmente por meio de hubs nacionais.



Essa hierarquia ajuda a explicar a densidade do grafo. Os aeroportos dos níveis 1 e 2 concentram grande parte das conexões, enquanto os níveis inferiores possuem conectividade decrescente.

5.3 Métricas Globais e por Região

Escopo	Ordem (	V	)	Tamanho (
Grafo Global	20	114	0,60	Alta conectividade
Nordeste	6	10	0,66	Acima da média
Sudeste	5	9	0,90	Alta concentração
Centro-Oeste	2	1	1,00	Grafo completo
Sul	3	2	0,66	Moderada
Norte	4	2	0,33	Menor conectividade

O grafo global possui 20 vértices e 114 arestas. Em um grafo simples não direcionado com 20 vértices, o número máximo possível de arestas é 190. Portanto, a densidade 0,60 indica que 60% das conexões possíveis estão presentes. Para uma rede de transporte aéreo simplificada, esse valor é alto.

A densidade elevada resulta principalmente da hierarquia de aeroportos. Os super hubs e hubs regionais se conectam a muitos aeroportos, gerando uma malha densa. Ao mesmo tempo, aeroportos intermediários, regionais e periféricos reduzem a densidade em relação a um grafo completo, o que torna a modelagem mais realista.

O Sudeste apresenta densidade interna de 0,90, o que reflete a forte concentração de aeroportos importantes e conexões entre eles. O Norte apresenta a menor densidade interna, 0,33, evidenciando menor conectividade regional dentro do recorte analisado. Essa diferença é coerente com a distribuição desigual da infraestrutura aérea brasileira.

5.4 Análise de Hubs

A análise de grau identificou BSB e CNF como os aeroportos de maior centralidade, ambos com grau 19. Em um grafo simples com 20 vértices, grau 19 significa que o aeroporto está conectado a todos os demais vértices da rede. Isso confirma o papel desses aeroportos como hubs fundamentais na estrutura modelada.

GRU também aparece entre os principais aeroportos da rede, reforçando a importância dos grandes centros nacionais. Esses hubs cumprem papel crítico porque reduzem distâncias em número de conexões e ajudam a manter a rede integrada. Em termos de Teoria dos Grafos, vértices de alto grau aumentam a conectividade global e tendem a reduzir o diâmetro efetivo da rede.

A existência desses hubs também mostra uma característica típica de redes de transporte: a conectividade não é distribuída uniformemente. Poucos vértices concentram grande parte das conexões, enquanto aeroportos regionais e periféricos dependem desses centros para alcançar o restante da rede.

5.5 Algoritmos Aplicados na Parte 1

Na Parte 1, BFS, DFS e Dijkstra foram aplicados ao grafo de aeroportos.

A BFS permite verificar a conectividade por camadas a partir de um aeroporto de origem. Ao partir de REC, o algoritmo alcança os aeroportos da rede por níveis de distância em número de arestas, o que permite observar quantas conexões são necessárias para chegar aos demais aeroportos. Como o grafo possui hubs altamente conectados, a rede apresenta alcance rápido a partir de aeroportos estratégicos.

A DFS também foi utilizada para explorar o grafo de aeroportos. Diferentemente da BFS, a DFS percorre um ramo até o final antes de retroceder. No contexto da Parte 1, sua principal utilidade foi confirmar a conectividade estrutural e comparar a ordem de visita com a BFS.

O Dijkstra foi aplicado aos pares de rotas definidos no projeto, utilizando os pesos de tempo de voo. Os resultados foram salvos em `out/distancias_rotas.csv` e visualizados na árvore de percurso.

Para o par REC → POA, o caminho mínimo encontrado foi direto:

Caminho: REC → POA - Custo total: 4,33 horas

Interpretação: a rota direta Recife–Porto Alegre é o caminho de menor custo no grafo, com duração aproximada de 4h20min. Como a aresta direta existe e possui menor custo acumulado que qualquer alternativa com escala, o Dijkstra seleciona corretamente o caminho direto.

Para o par GRU → MAO, o caminho mínimo também foi direto:

Caminho: GRU → MAO - Custo total: 3,83 horas

Interpretação: a rota Guarulhos–Manaus é servida por conexão direta, com duração aproximada de 3h50min. O Dijkstra confirma que nenhuma rota intermediária apresenta custo acumulado menor que o voo direto.

Esses dois resultados demonstram que o algoritmo não força caminhos com intermediários quando a rota direta é a melhor opção. Isso é importante porque valida tanto a implementação quanto a coerência dos pesos definidos.

5.6 Visualização da Parte 1



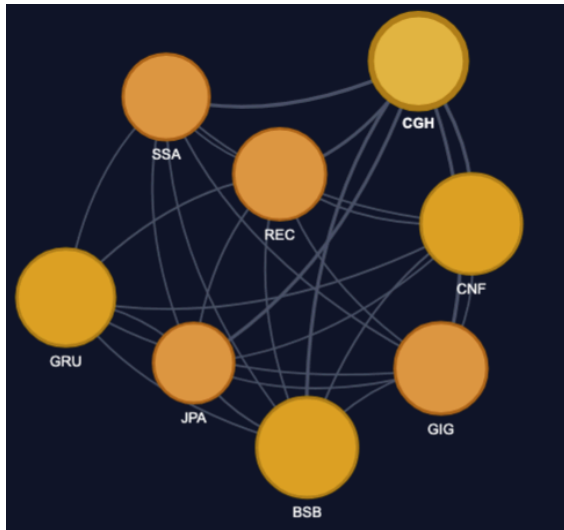
Grafo interativo da malha aérea brasileira, com aeroportos representados como nós e rotas diretas como arestas.

Discussão: A visualização permite observar a concentração de conexões nos principais hubs. O tamanho dos nós é proporcional ao grau, facilitando a identificação visual de aeroportos centrais como BSB, CNF e GRU. O layout baseado em forças evidencia a estrutura densa da rede e a dependência dos aeroportos periféricos em relação aos hubs.



Árvore de percurso dos caminhos mínimos calculados pelo Dijkstra para os pares REC → POA e GRU → MAO.

Discussão: A árvore de percurso mostra que ambos os caminhos obrigatórios são conexões diretas. Isso reforça que o Dijkstra seleciona o menor custo acumulado, e não necessariamente o caminho com mais etapas. Quando uma aresta direta possui o menor peso total, ela é corretamente escolhida.



Subgrafo dos principais hubs da rede de aeroportos.

Discussão: O subgrafo isola os aeroportos de maior grau e evidencia a formação de um núcleo altamente conectado. Esse núcleo é responsável por sustentar grande parte da conectividade global do grafo. A retirada de hubs desse tipo teria impacto relevante sobre a eficiência e a robustez da rede.

5.7 Arquivos de Saída da Parte 1

Arquivo	Conteúdo
out/global.json	Métricas globais do grafo: ordem, tamanho e densidade
out/regioes.json	Métricas estruturais por região geográfica
out/graus.csv	Grau de cada aeroporto
out/ego_aeroportos.csv	Densidade da ego-network de cada aeroporto
out/rankings.json	Ranking dos aeroportos mais conectados
out/distancias_rotas.csv	Caminhos mínimos calculados pelo Dijkstra
out/grafico_interativo.html	Visualização interativa do grafo completo
out/arvore_percurso.html	Árvore de percurso dos caminhos calculados
out/subgrafo_hubs.html	Subgrafo dos principais hubs

out/histograma.png	Histograma de distribuição de graus
out/ranking.png	Gráfico de ranking dos aeroportos
out/regioes.png	Comparativo regional de conectividade
out/avd_dashboard.html	Dashboard analítico de visualização de dados

6. Parte 2 — Rede de Filmes IMDb

6.1 Objetivo da Parte 2

O objetivo da Parte 2 foi modelar uma rede de filmes como grafo não direcionado e ponderado, permitindo analisar relações de similaridade entre obras, conectividade da rede, presença de hubs e caminhos entre filmes. A proposta se aproxima da lógica de sistemas de recomendação, nos quais conteúdos são aproximados por características compartilhadas.

6.2 Descrição do Dataset

O dataset final da Parte 2 foi organizado em dois arquivos principais:

Arquivo	Entidade	Colunas
Imdb_filmes.csv	Vértices	filme, ano, generos, nota, diretores
Imdb_arestas.csv	Arestas	filme1, filme2, qtd_atores_compartilhados, atores_compartilhados, generos_compartilhados, similaridade, peso

O arquivo Imdb_filmes.csv define os vértices da rede. Cada linha representa um filme com seus atributos principais. O arquivo Imdb_arestas.csv define as conexões entre filmes. Cada linha representa uma aresta entre dois filmes, acompanhada da quantidade de atores compartilhados, gêneros compartilhados, similaridade calculada e peso utilizado nos algoritmos.

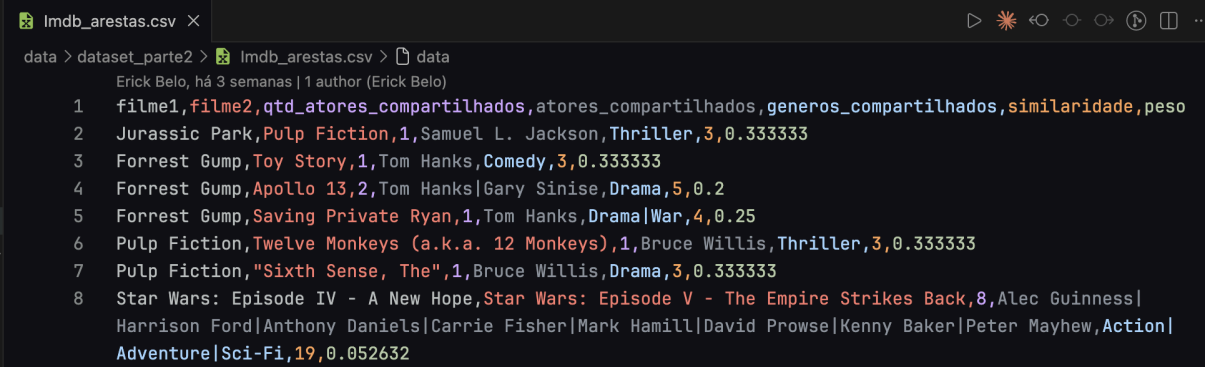
A construção do dataset priorizou filmes reconhecíveis e conexões com significado interpretável. A similaridade foi calculada com base em atores/atrizes e gêneros compartilhados. A coluna de diretores foi mantida como atributo informativo dos filmes, mas não foi usada na fórmula final de similaridade, pois poderia criar conexões pouco representativas em alguns casos.

6.3 Modelagem do Grafo

6.3.1 Vértices

Cada filme é representado como um vértice do grafo. Essa escolha é coerente com o objetivo da Parte 2, pois o foco da análise é a relação entre obras cinematográficas. Uma alternativa possível seria construir um grafo bipartido entre filmes e atores, mas essa abordagem mudaria o problema: em vez de medir similaridade entre filmes, o grafo passaria a representar participação de atores em obras.

Ao usar filmes como vértices, a rede se torna mais próxima de um sistema de recomendação, em que o usuário deseja navegar de um filme para outro por critérios de proximidade.



```
data > dataset_parte2 > lmdb_arestas.csv > data
Erick Belo, há 3 semanas | 1 author (Erick Belo)
1 filme1,filme2,qtd_atores_compartilhados,atores_compartilhados,generos_compartilhados,similaridade,peso
2 Jurassic Park,Pulp Fiction,1,Samuel L. Jackson,Thriller,3,0.333333
3 Forrest Gump,Toy Story,1,Tom Hanks,Comedy,3,0.333333
4 Forrest Gump,Apollo 13,2,Tom Hanks|Gary Sinise,Drama,5,0.2
5 Forrest Gump,Saving Private Ryan,1,Tom Hanks,Drama|War,4,0.25
6 Pulp Fiction,Twelve Monkeys (a.k.a. 12 Monkeys),1,Bruce Willis,Thriller,3,0.333333
7 Pulp Fiction,"Sixth Sense, The",1,Bruce Willis,Drama,3,0.333333
8 Star Wars: Episode IV - A New Hope,Star Wars: Episode V - The Empire Strikes Back,8,Alec Guinness|
Harrison Ford|Anthony Daniels|Carrie Fisher|Mark Hamill|David Prowse|Kenny Baker|Peter Mayhew>Action|
Adventure|Sci-Fi,19,0.052632
```

6.3.2 Arestas

Uma aresta conecta dois filmes quando eles compartilham atores e/ou gêneros. Como a relação de similaridade é simétrica, o grafo foi modelado como não direcionado. Se o filme A é similar ao filme B, então o filme B também é similar ao filme A pelo mesmo critério.

Essa modelagem é adequada porque não existe uma direção natural na relação. Diferentemente de uma rede de citações ou dependências, em que uma aresta pode ter sentido, a similaridade entre filmes é uma relação bilateral.

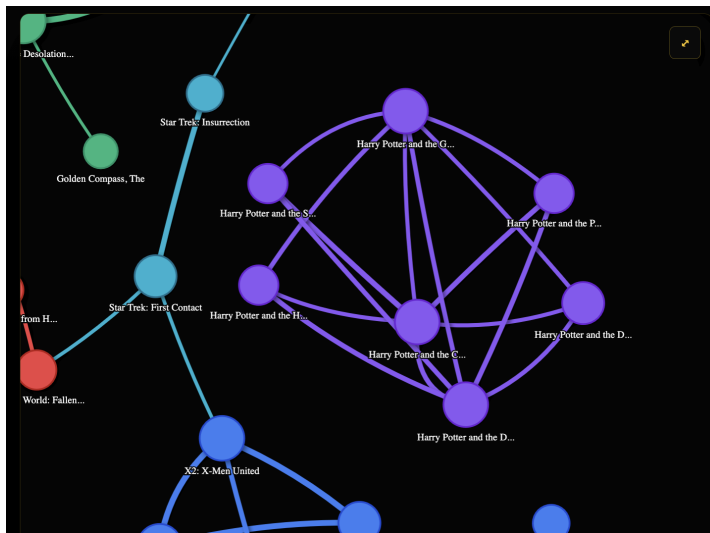
6.3.3 Definição da Similaridade

A similaridade entre dois filmes foi definida pela fórmula:

$$\text{similaridade} = (2 \times \text{atores em comum}) + \text{gêneros em comum}$$

O fator 2 atribuído aos atores foi escolhido porque compartilhar um ator é uma evidência mais específica de proximidade do que compartilhar apenas um gênero. Gêneros como drama, ação ou comédia são categorias amplas e conectam muitos filmes de forma genérica. Já um ator compartilhado cria um vínculo mais concreto entre obras, podendo indicar proximidade de elenco, público e estilo.

Essa fórmula é uma heurística simples, mas adequada ao escopo do projeto. Ela permite transformar atributos qualitativos em um valor numérico que pode ser usado na construção dos pesos do grafo.



6.3.4 Definição dos Pesos

O peso da aresta foi definido como:

$$\text{peso} = 1 / \text{similaridade}$$

Essa escolha inverte a relação entre similaridade e custo. Quanto maior a similaridade entre dois filmes, menor o peso da aresta. Isso é necessário porque o Dijkstra encontra caminhos de menor custo. Assim, ao minimizar a soma dos pesos, o algoritmo tende a escolher sequências de filmes mais semelhantes entre si.

Essa definição também preserva a pré-condição do Dijkstra, pois os pesos são sempre positivos. Na prática, o dataset final foi construído com arestas cuja similaridade é maior que zero. Dessa forma, não há divisão zero e todas as arestas possuem peso positivo.

6.4 Propriedades Estruturais do Grafo

Métrica	Valor
Vértices	3.899
Arestas	63.484
Tipo do grafo	Não direcionado, ponderado e com pesos positivos
Componentes conexas	1
Maior componente conexa	3.899 filmes
Percentual na maior componente	100%
Grau médio	aproximadamente 32,6
Maior grau registrado	aproximadamente 142

O grafo IMDb final possui 3.899 filmes e 63.484 conexões. Embora seja muito maior que o grafo de aeroportos, sua densidade relativa é menor, aproximadamente 0,0084. Isso é

esperado em grafos grandes: mesmo com dezenas de milhares de arestas, a proporção de conexões existentes em relação ao total possível tende a ser pequena.

A existência de uma única componente conexa é um resultado relevante. Ela indica que todos os filmes do dataset estão conectados entre si por alguma sequência de relações de ator ou gênero. Em termos práticos, isso significa que, partindo de qualquer filme, é possível alcançar qualquer outro filme do grafo por um caminho.

O grau médio de aproximadamente 32,6 indica que cada filme está conectado, em média, a cerca de 33 outros filmes. Esse valor mostra que a rede possui conectividade suficiente para permitir navegação eficiente, mas ainda preserva diferenças entre filmes mais periféricos e filmes hubs.

6.5 Interpretação da Componente Única

O fato de o grafo possuir apenas uma componente conexa indica alta coesão estrutural. Isso significa que não existem grupos isolados de filmes dentro do dataset final. Mesmo filmes que não possuem relação direta podem ser conectados por intermediários.

Esse resultado é importante para a interface e para o roteamento com Dijkstra. Como todos os vértices pertencem à mesma componente, qualquer par de filmes presente no dataset tem algum caminho possível. Assim, a busca de caminhos mínimos passa a depender principalmente dos pesos e da similaridade, e não da existência ou ausência de conexão estrutural.

6.6 Interpretação do Grau Médio e dos Hubs

O grau médio de aproximadamente 32,6 mostra que a rede possui quantidade considerável de conexões por filme. No entanto, a média não significa que todos os filmes tenham conectividade parecida. Em redes reais, é comum que poucos vértices tenham grau muito alto, enquanto a maioria possui grau mais próximo ou abaixo da média.

Na rede IMDb, os filmes com maior grau funcionam como hubs culturais. Eles compartilham atores ou gêneros com muitos outros filmes e, por isso, ocupam posição central na navegação da rede. Esses hubs reduzem distâncias entre filmes e aumentam a coesão global do grafo.

7. Algoritmos Implementados

Todos os algoritmos foram implementados manualmente em Python, usando uma estrutura de grafo baseada em lista de adjacência com pesos. Essa representação é eficiente para grafos esparsos e adequada aos dois casos do projeto.

7.1 BFS — Busca em Largura

A complexidade da BFS é $O(V + E)$, pois cada vértice e cada aresta são processados no máximo uma vez.

Na Parte 2, a BFS foi usada para explorar a rede de filmes a partir de um filme inicial, como Jurassic Park. Como o grafo possui uma única componente conexa, a BFS alcança todos os 3.899 filmes do dataset. A análise por camadas é útil para interpretar graus de separação entre filmes, isto é, quantas relações sucessivas de similaridade são necessárias para conectar uma obra a outra.

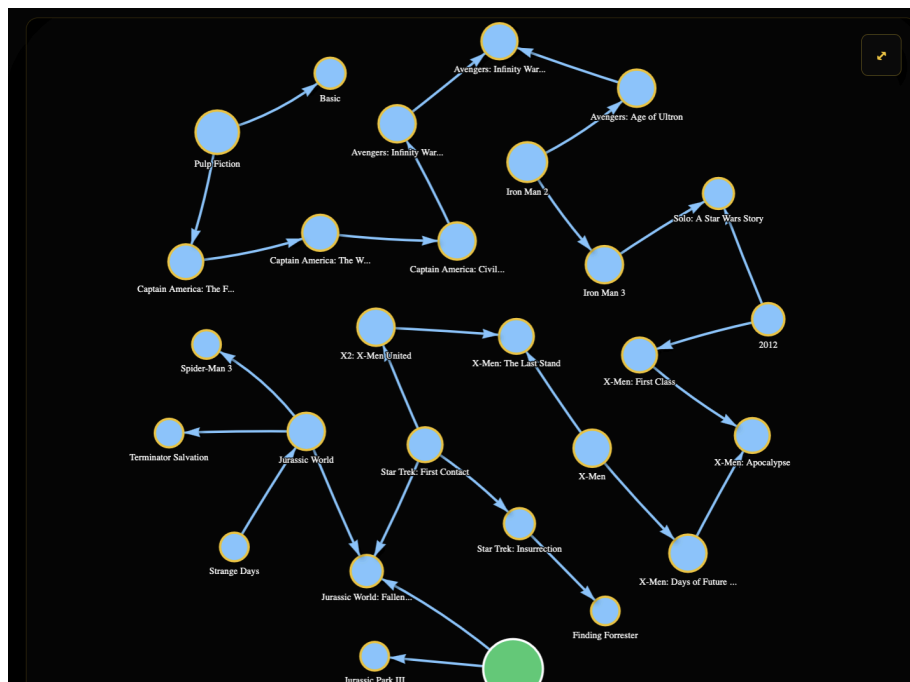
7.2 DFS — Busca em Profundidade

A DFS, ou Depth-First Search, explora o grafo avançando por um ramo até o máximo possível antes de retroceder. A implementação principal utiliza uma pilha explícita, seguindo a lógica LIFO.

A complexidade da DFS também é $O(V + E)$, pois cada vértice e cada aresta são examinados em tempo linear em relação ao tamanho do grafo.

Na Parte 1, a DFS foi usada para explorar a estrutura do grafo de aeroportos e confirmar a conectividade. Na Parte 2, a DFS também alcança todos os 3.899 filmes, pois a rede possui componentes únicos. A diferença em relação à BFS está na ordem de visita: enquanto a BFS avança por camadas, a DFS tende a aprofundar uma sequência de conexões antes de retornar.

A implementação `dfs_classificacao_dirigido` foi validada separadamente para classificação de arestas em grafos dirigidos, incluindo arestas de árvore, retorno, avanço e cruzamento. Já nos grafos não direcionados principais do projeto, a DFS iterativa foi usada principalmente para exploração e conectividade, não como mecanismo principal de detecção de ciclos.



7.3 Dijkstra — Caminho Mínimo Ponderado

O algoritmo de Dijkstra calcula o caminho de menor custo acumulado a partir de um vértice inicial. Ele utiliza uma fila de prioridade para sempre expandir o vértice com menor distância conhecida.

A complexidade com o heap binário é $O((V + E) \log V)$. Essa complexidade é maior que a de BFS e DFS, mas o algoritmo resolve um problema mais específico: encontrar caminhos mínimos considerando pesos.

Na Parte 1, Dijkstra foi usado para calcular rotas de menor tempo entre aeroportos. Como os pesos representam duração estimada de voo, o caminho mínimo corresponde à rota com menor tempo total.

Na Parte 2, Dijkstra foi usado para encontrar caminhos de maior proximidade entre filmes. Como o peso é $1/\text{similaridade}$, minimizar o custo total significa favorecer sequências de filmes com maior similaridade acumulada. Esse uso é coerente com a ideia de navegação semântica entre obras.

A principal pré-condição do Dijkstra é que os pesos não sejam negativos. Essa condição é atendida nas duas partes do projeto: os tempos de voo são positivos e os pesos da rede de filmes são positivos por construção.

7.4 Bellman-Ford — Pesos Negativos e Ciclos Negativos

O Bellman-Ford calcula caminhos mínimos a partir de uma origem mesmo quando existem pesos negativos. Ele realiza $V - 1$ rodada de relaxamento sobre todas as arestas e, ao final, verifica se ainda existe alguma distância que pode ser reduzida. Se isso ocorrer, há ciclo negativo alcançável a partir da origem.

A complexidade do Bellman-Ford é $O(V \times E)$, superior à do Dijkstra para grafos com pesos não negativos. Por isso, ele não foi aplicado ao grafo principal de filmes, que possui apenas pesos positivos. Nesse caso, usar Bellman-Ford seria menos eficiente e não acrescentaria valor técnico à análise principal.

Para demonstrar corretamente o algoritmo, foram usados grafos artificiais específicos.

O primeiro grafo artificial, `bf_validacao_sem_ciclo.csv`, é um grafo dirigido com 4 vértices: Z, a, b e c. Suas arestas são $Z \rightarrow a$ com peso +5, $Z \rightarrow b$ com peso +2, $a \rightarrow c$ com peso -3 e $b \rightarrow c$ com peso +1. O resultado esperado é $Z = 0$, $a = 5$, $b = 2$ e $c = 2$. Esse resultado ocorre porque o caminho $Z \rightarrow a \rightarrow c$ tem custo $5 + (-3) = 2$, menor que o caminho $Z \rightarrow b \rightarrow c$, cujo custo é 3.

O segundo grafo artificial, `bf_validacao_com_ciclo.csv`, é um grafo dirigido com 3 vértices: a, b e c. Suas arestas são $a \rightarrow b$ com peso +1, $b \rightarrow c$ com peso +1 e $c \rightarrow a$ com peso -3. O ciclo $a \rightarrow b \rightarrow c \rightarrow a$ tem custo total -1, caracterizando ciclo negativo. O algoritmo detecta corretamente esse caso, retornando `ciclo_negativo=True`.

Algoritmo	Complexidade	Pesos Negativos	Grafo Usado	Principal Resultado
BFS	$O(V + E)$	Não aplicável	Aeroportos e filmes	Alcance por camadas
DFS	$O(V + E)$	Não aplicável	Aeroportos e filmes	Exploração em profundidade e conectividade

Dijkstra	$O((V + E) \log V)$	Não aceita pesos negativos	Aeroportos e filmes	Caminho mínimo ponderado
Bellman-Ford	$O(V \times E)$	Aceita pesos negativos	Grafos artificiais	Pesos negativos e detecção de ciclo negativo

8. Resultados e Análises

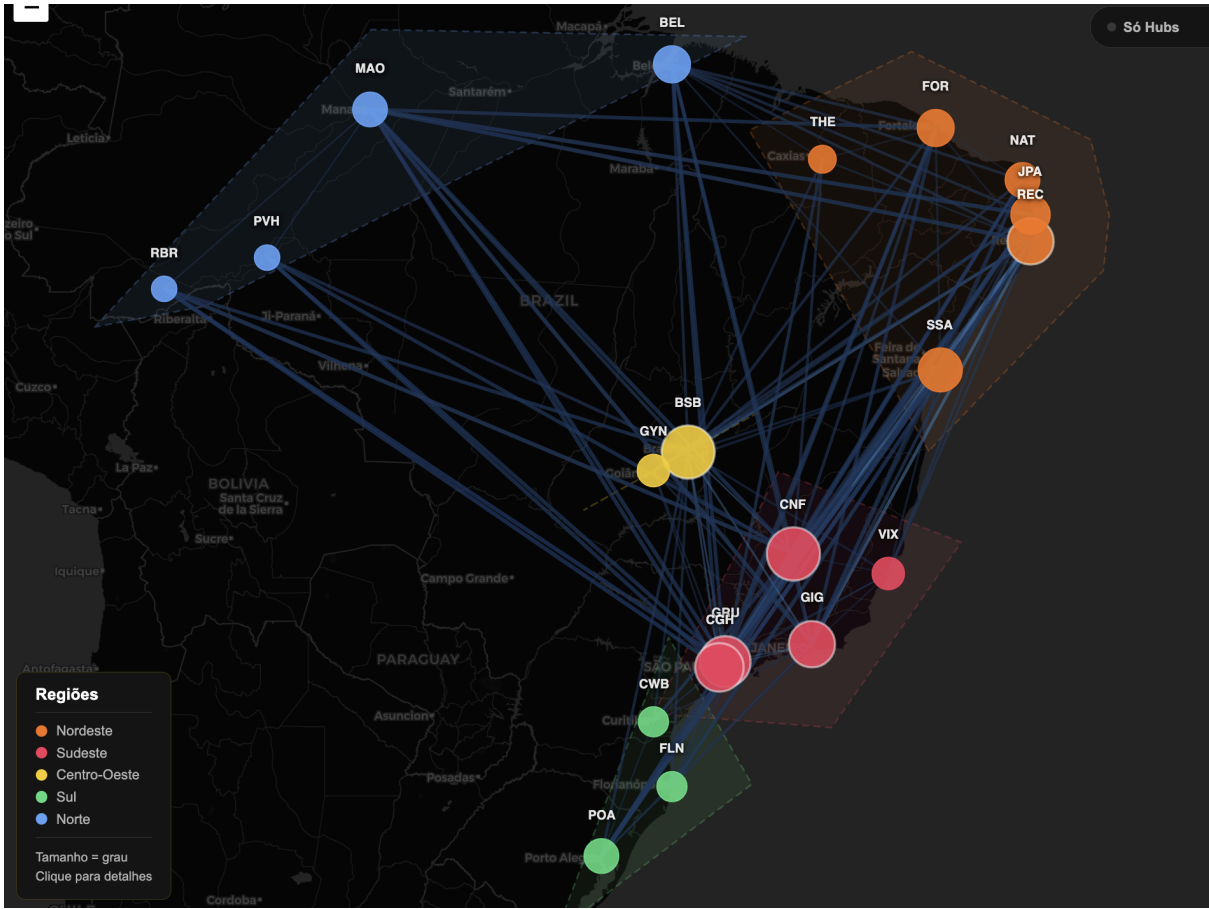
8.1 Resultados da Parte 1

Métrica	Valor
Ordem (	V
Tamanho (	E
Densidade global	0,60
Aeroportos com maior grau	BSB e CNF — grau 19
Região de maior densidade interna	Centro-Oeste — densidade 1,00
Região de menor densidade interna	Norte — densidade 0,33
Caminho mínimo REC → POA	REC → POA, custo 4,33h
Caminho mínimo GRU → MAO	GRU → MAO, custo 3,83h

A Parte 1 mostra uma rede pequena, porém altamente conectada. A densidade 0,60 é elevada para uma rede de transporte e decorre da presença de hubs muito conectados. BSB e CNF, ambos com grau 19, são os vértices mais centrais do grafo, conectando-se a todos os demais aeroportos.

Os caminhos mínimos obrigatórios retornaram às conexões diretas. Isso é coerente com o domínio do problema, pois voos diretos tendem a ter menor tempo total que rotas com escalas. O resultado também mostra que o Dijkstra foi aplicado corretamente: ele não busca

o caminho com mais conexões, mas o menor custo acumulado.



8.2 Resultados da Parte 2

Métrica	Valor
Vértices	3.899
Arestas	63.484
Componentes conexas	1
Maior componente conexa	3.899 filmes
Percentual na maior componente	100%
Grau médio	aproximadamente 32,6
Maior grau registrado	aproximadamente 142
BFS a partir de filme fonte	alcança todos os 3.899 filmes
DFS a partir de filme fonte	alcança todos os 3.899 filmes
Bellman-Ford sem ciclo negativo	Z=0, a=5, b=2, c=2
Bellman-Ford com ciclo negativo	ciclo_negativo=True
Testes automatizados	13 de 13 aprovados

O resultado mais importante da Parte 2 é a existência de uma única componente conexa. Isso significa que a rede de filmes é totalmente conectada no dataset final. Mesmo que dois

filmes não tenham atores ou gêneros diretamente em comum, existe uma sequência de filmes intermediários que conecta ambos.

A BFS e a DFS confirmam essa propriedade ao alcançar todos os 3.899 vértices a partir de um filme fonte. A BFS é mais adequada para estudar a distância em camadas, enquanto a DFS é mais adequada para explorar profundidade e estrutura de percurso.

O Dijkstra permite transformar essa conectividade em navegação ponderada. Em vez de apenas encontrar qualquer caminho entre dois filmes, ele busca o caminho de menor custo, favorecendo conexões de maior similaridade.

8.3 Benchmark de Desempenho

Algoritmo	Tempo registrado (s)	Grafo	Observação
BFS	0,089	Grafo IMDb	Exploração linear
DFS	0,106	Grafo IMDb	Exploração linear
Dijkstra	0,132	Grafo IMDb	Usa heap de prioridade
Bellman-Ford	0,000037	Grafos artificiais	Não comparável diretamente

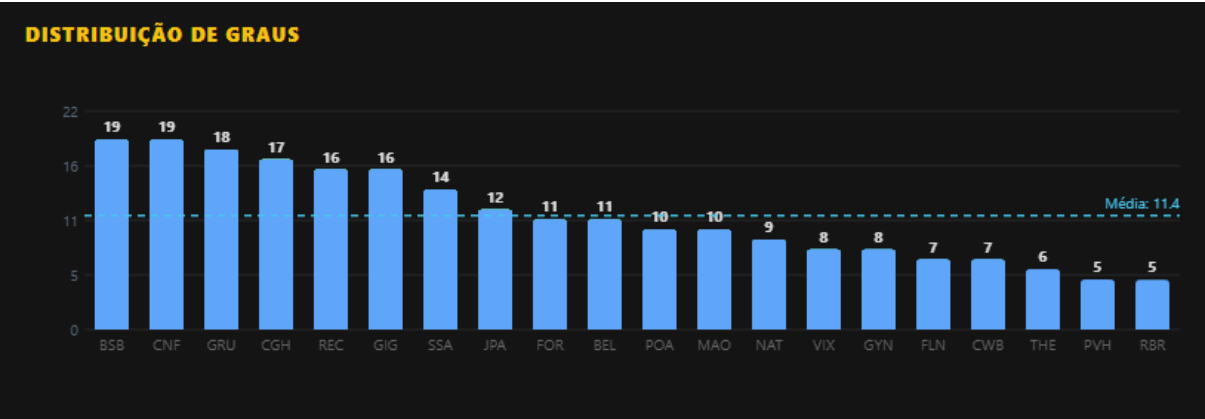
Os tempos apresentados correspondem a execuções registradas no ambiente de desenvolvimento. Como foram obtidos por execução única, devem ser interpretados como referência prática, não como benchmark estatístico definitivo.

A relação entre os tempos é coerente com a teoria. BFS e DFS são os mais rápidos porque têm complexidade $O(V + E)$. Dijkstra é mais custoso por utilizar uma fila de prioridade e calcular distâncias ponderadas. Bellman-Ford aparece com tempo muito baixo porque foi executado em grafos artificiais pequenos; por isso, seu tempo não deve ser comparado diretamente ao dos algoritmos executados no grafo IMDb.

9. Visualizações e Interface Web

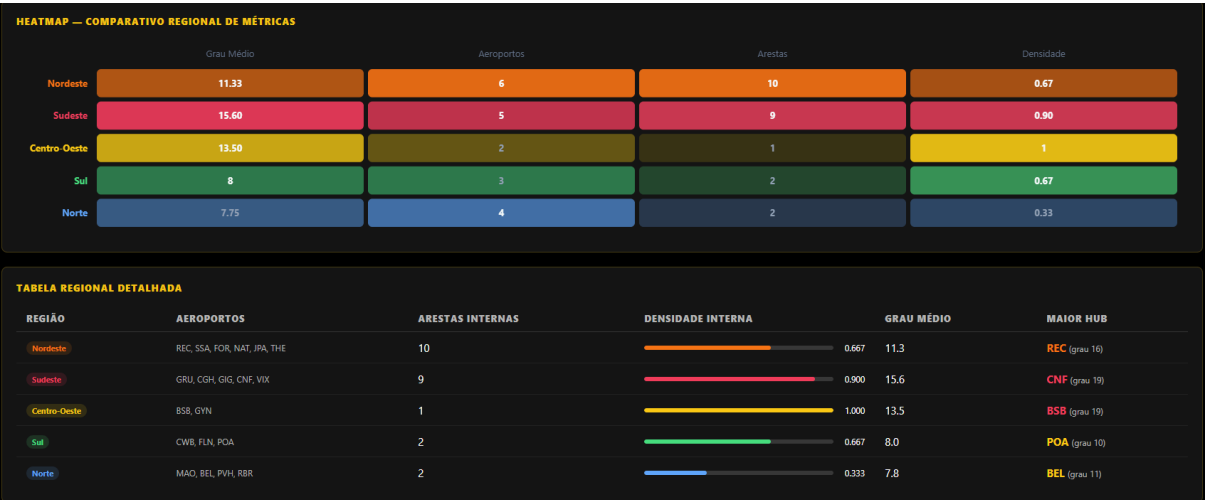
9.1 Visualizações da Parte 1

A Parte 1 gerou visualizações interativas e estáticas para facilitar a interpretação da rede de aeroportos. O grafo interativo mostra todos os aeroportos e rotas diretas. A árvore de percurso destaca os caminhos mínimos calculados pelo Dijkstra. O subgrafo de hubs isola os aeroportos mais conectados. Os gráficos estáticos complementam a análise com distribuição de graus, ranking de conexões e comparação regional.



Ranking dos aeroportos com maior número de conexões.

Discussão: O ranking confirma numericamente a centralidade dos hubs. Aeroportos com maior grau aparecem como pontos fundamentais para a conectividade da rede. A presença de poucos aeroportos muito conectados e vários aeroportos menos conectados evidencia uma distribuição desigual de centralidade.



Comparação regional de conectividade da rede de aeroportos.

Discussão: A comparação regional mostra que a conectividade não é homogênea entre as regiões brasileiras analisadas. Sudeste e Centro-Oeste apresentam alta densidade interna dentro do recorte, enquanto o Norte aparece com menor densidade. Isso reforça que a estrutura da malha aérea é influenciada por concentração econômica, geográfica e operacional.

9.2 Visualizações da Parte 2

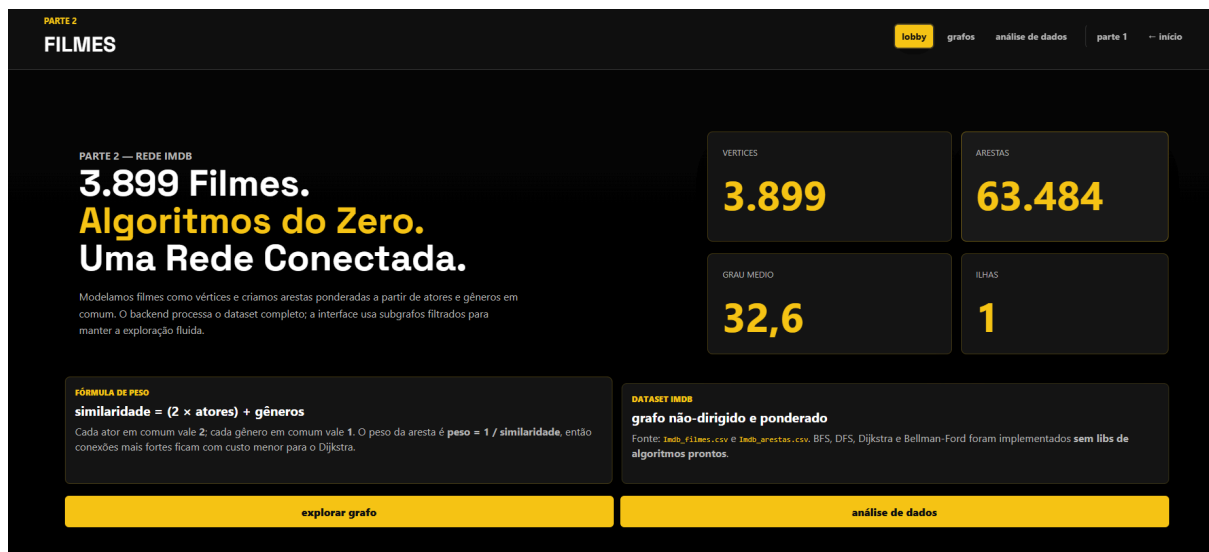
A Parte 2 gerou visualizações específicas para análise da rede IMDb. Os principais arquivos de visualização são:

parte2_estrutura_rede.png: dashboard com distribuição de graus, top hubs e conectividade.

parte2_heatmap_bfs_dfs.png: comparação entre os primeiros nós visitados por BFS e DFS.

parte2_caminho_dijkstra.png: visualização de caminhos mínimos entre filmes.

parte2_heatmap_performance.png: comparação de adequação e desempenho dos algoritmos.



Dashboard exploratório da estrutura da rede IMDb.

Discussão: O dashboard permite observar a distribuição de graus e a presença de hubs. A rede não apresenta distribuição homogênea: alguns filmes concentram muitas conexões, enquanto outros possuem grau menor. Essa assimetria é típica de redes reais.

9.3 Interface Web

A interface web foi desenvolvida em HTML, CSS e JavaScript puro. A biblioteca `vis-network` foi usada localmente para renderização interativa do grafo. A opção por tecnologias simples reduz dependências e facilita a execução pelo avaliador, pois a interface pode ser aberta por meio de um servidor HTTP local simples.

A interface consome arquivos JSON pré-processados, evitando que o usuário precise executar todo o pipeline Python para explorar os resultados.

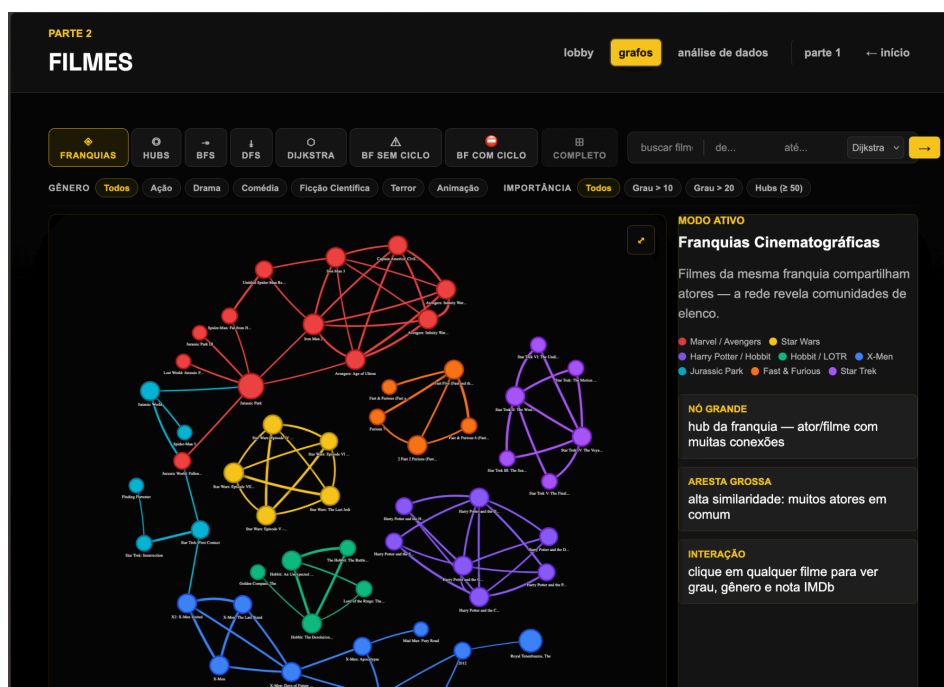
Arquivo JSON	Conteúdo
<code>resumo_parte2.json</code>	Métricas, resultados de algoritmos e benchmark
<code>parte2_amostra.json</code>	Amostra visual de filmes e arestas para renderização
<code>parte2_grafo.json</code>	Grafo completo com 3.899 vértices e 63.484 arestas

É importante destacar que a interface web da Parte 2 **não foi construída em** `src/viz.py` nem em `src/partel1/viz.py`. Esses módulos são responsáveis exclusivamente pelas saídas estáticas da Parte 1: gráficos Matplotlib (histograma, ranking, regiões, subgrafo de hubs) e HTMLs Pyvis do grafo interativo de aeroportos. A geração de todos os dados e visualizações consumidos pela interface da

Parte 2 é feita pelos arquivos `src/parte2/build_interface_data.py` (JSONs de métricas e amostra), `src/parte2/build_interface_grafo.py` (JSON do grafo completo) e `src/parte2/build_visualizations.py` (PNGs analíticos em `interface/assets/`). O frontend em `interface/` consome diretamente esses artefatos.

A interface inclui modos de visualização para maior similaridade, top conexões, BFS, DFS, Dijkstra, Bellman-Ford e grafo completo. Também possui autocomplete para seleção de filmes, permitindo ao usuário escolher origem e destino para o roteamento.

A renderização do grafo completo é uma limitação prática, pois exibir 63.484 arestas simultaneamente pode sobrecarregar navegadores comuns. Por isso, a visualização trabalha com uma amostra mais leve, enquanto o roteamento continua utilizando os dados completos do grafo.



Interface web da rede IMDb com seleção de filmes.

Discussão: A interface torna os resultados do projeto mais acessíveis. Em vez de depender apenas de saídas em terminal ou arquivos JSON, o usuário pode explorar visualmente a rede, buscar filmes e calcular caminhos entre obras.

10. Testes e Validação

10.1 Cobertura de Testes

Foram implementados 13 testes automatizados com `pytest`. Os testes foram construídos com grafos pequenos e controlados, o que facilita a verificação determinística dos resultados. Essa estratégia é adequada porque valida os algoritmos independentemente

dos datasets reais, evitando que erros nos dados sejam confundidos com erros de implementação.

Algoritmo	Cenário Testado	O que Valida	Importância
BFS	Níveis corretos em grafo pequeno	Ordem de visita por camadas	Corretude básica da busca
BFS	Componente isolada	Vértices inacessíveis não são visitados	Robustez em grafo desconexo
BFS	Níveis em cadeia	Propagação correta por distância	Validação de camadas
DFS	Percurso em profundidade	Ordem de visita por pilha	Corretude estrutural
DFS	Classificação de arestas	Árvore, retorno, avanço e cruzamento	Análise em grafo dirigido
DFS	Ciclo em grafo dirigido	Identificação de aresta de retorno	Validação conceitual
Dijkstra	Caminho correto em grafo simples	Distâncias e predecessores	Corretude do caminho mínimo
Dijkstra	Ausência de caminho	Retorno nulo entre componentes	Robustez
Dijkstra	Rejeição de peso negativo	Respeito à pré-condição	Segurança algorítmica
Dijkstra	Variação parametrizada de peso negativo	Tratamento consistente de entrada inválida	Cobertura adicional
Bellman-Ford	Peso negativo sem ciclo negativo	Distâncias corretas	Caso central do algoritmo
Bellman-Ford	Ciclo negativo	ciclo_negativo=True	Propriedade exclusiva do algoritmo
I/O	Carregamento de CSV	Parsing de vértices, arestas e pesos	Integridade do pipeline

A aprovação dos 13 testes indica que os algoritmos foram validados em cenários controlados antes de serem aplicados aos datasets reais. Essa etapa aumenta a confiabilidade dos resultados e demonstra preocupação com corretude, não apenas com visualização.

11. Decisões Técnicas do Grupo

Decisão	Justificativa Técnica	Trade-off
Implementar algoritmos do zero	Atende ao objetivo da disciplina e demonstra compreensão real das estruturas de dados e algoritmos	Maior esforço de desenvolvimento
Usar lista de adjacência	Representação eficiente para grafos esparsos e adequada para iteração sobre vizinhos	Menos direta para verificar existência de aresta específica
Usar grafo não direcionado na Parte 1	A análise foca conectividade estrutural entre aeroportos, não diferença operacional por sentido	Não captura assimetrias reais de frequência ou demanda
Usar grafo não direcionado na Parte 2	Similaridade entre filmes é relação simétrica	Não modela relações direcionais de recomendação
Usar tempo de voo como peso na Parte 1	Permite interpretar caminho mínimo como menor duração total	Não considera preço, escala, espera ou frequência
Usar peso = 1/similaridade na Parte 2	Permite ao Dijkstra favorecer conexões mais similares	Custo tem interpretação inversa à similaridade
Validar Bellman-Ford em grafos artificiais	O grafo principal tem pesos positivos; grafos artificiais demonstram pesos negativos e ciclos negativos de forma correta	Bellman-Ford não é aplicado ao dataset principal
Usar HTML/CSS/JS puro na interface	Reduz dependências e facilita execução local	Menos recursos prontos de framework
Usar vis-network local	Permite visualização interativa sem depender de CDN	Aumenta levemente o tamanho do projeto
Renderizar amostra visual do grafo	Evita travamentos no navegador	A visualização não mostra todas as arestas simultaneamente

Essas decisões mostram que o projeto priorizou correção, reprodutibilidade e clareza. Algumas escolhas reduzem o impacto visual ou sofisticação da interface, mas aumentam a confiabilidade acadêmica da entrega.

12. Limitações

12.1 Limitações da Parte 1

A rede de aeroportos possui apenas 20 aeroportos, o que representa uma fração da malha aérea brasileira real. A seleção permite uma análise clara e controlada, mas não representa toda a complexidade do sistema aéreo nacional.

Os pesos foram definidos a partir de tempos estimados de voo direto. Esses valores podem variar em relação a dados oficiais de companhias aéreas ou bases regulatórias. Além disso, o modelo não considera tempo de conexão, atraso, preço, frequência de voos, demanda, capacidade de aeronaves ou disponibilidade por horário.

A modelagem não direcionada também simplifica o domínio. Na prática, uma rota pode ter maior frequência em um sentido do que no outro, ou pode existir sazonalidade. Esses fatores não foram representados.

12.2 Limitações da Parte 2

A fórmula de similaridade é uma heurística baseada em atores e gêneros. Ela não captura fatores como idioma, país, época, tom, subgênero, avaliação de usuários, sinopse, estilo visual ou recepção crítica.

O peso atribuído a atores/atrizes e gêneros é fixo. Na prática, alguns gêneros são mais genéricos que outros, e alguns atores aparecem em filmes muito diversos. Uma modelagem mais sofisticada poderia usar ponderação por frequência, como IDF, para reduzir o peso de atributos muito comuns.

O dataset pode conter vieses, como maior presença de filmes populares, anglófonos ou de determinados gêneros. Isso afeta a estrutura da rede e a interpretação dos hubs.

A interface precisa limitar a quantidade de arestas renderizadas para evitar problemas de desempenho no navegador. Portanto, a visualização interativa não exibe simultaneamente todas as conexões do grafo completo, embora os dados completos sejam utilizados para roteamento.

12.3 Limitações dos Algoritmos

BFS e DFS não consideram pesos. Assim, seus resultados são úteis para conectividade e ordem de visita, mas não para custo ponderado.

Dijkstra não aceita pesos negativos. Essa limitação foi respeitada no projeto, pois os grafos principais possuem apenas pesos positivos.

Bellman-Ford aceita pesos negativos e detecta ciclos negativos, mas possui complexidade $O(V \times E)$, tornando-se menos eficiente para grafos grandes. Por isso, foi validado em grafos artificiais específicos.

12.4 Limitações do Benchmark

Os tempos de execução registrados dependem do hardware, do sistema operacional, da carga da máquina e de detalhes de implementação. Como os valores são registros de execução no ambiente do projeto, eles devem ser interpretados como evidência prática de funcionamento, não como medição estatística rigorosa.

13. Conclusão

O Projeto Final de Teoria dos Grafos atingiu seus objetivos técnicos e acadêmicos. Dois problemas reais foram modelados formalmente como grafos: uma rede de aeroportos brasileiros e uma rede de filmes baseada em similaridade. Em ambos os casos, as escolhas de vértices, arestas e pesos foram justificadas de acordo com o domínio.

Na Parte 1, o grafo de aeroportos apresentou 20 vértices, 114 arestas e densidade 0,60. A análise identificou BSB e CNF como hubs de grau 19, evidenciando a importância desses aeroportos para a conectividade da rede. O Dijkstra calculou corretamente os caminhos mínimos $REC \rightarrow POA$ e $GRU \rightarrow MAO$, ambos diretos, com custos de 4,33h e 3,83h, respectivamente.

Na Parte 2, o grafo de filmes apresentou 3.899 vértices, 63.484 arestas e uma única componente conexa. Isso demonstra que todos os filmes do dataset final estão conectados por alguma sequência de similaridades. O grau médio de aproximadamente 32,6 e a presença de hubs mostram que a rede possui estrutura típica de grafos reais, com conectividade desigual e vértices centrais.

Os quatro algoritmos exigidos foram implementados manualmente em Python puro. BFS e DFS foram usados para exploração e conectividade; Dijkstra foi usado para caminhos mínimos ponderados; e Bellman-Ford foi validado em grafos artificiais com pesos negativos e ciclo negativo. A aprovação dos 13 testes automatizados reforça a confiabilidade das implementações.

As visualizações e a interface web complementam a análise ao transformar os resultados algorítmicos em representações compreensíveis e navegáveis. Mesmo com limitações de escopo, heurística e desempenho visual, o projeto demonstra de forma consistente como a Teoria dos Grafos pode ser aplicada a problemas reais de transporte, recomendação e análise de redes.

Portanto, o trabalho evidencia não apenas a execução dos algoritmos, mas também a capacidade de modelar problemas, justificar decisões técnicas, interpretar métricas e reconhecer limitações. Essa combinação de implementação, análise e documentação representa o principal valor acadêmico do projeto.

TEORIA DOS GRAFOS

ANÁLISE E VISUALIZAÇÃO DE DADOS

Redes, Algoritmos e Visualizações

Modelagem de grafos aplicada a aeroportos brasileiros e à rede de filmes do IMDb, com análise comparativa de algoritmos de busca e caminho mínimo, e análise e visualização de dados para revelar a estrutura de cada rede.

NÚMEROS DO PROJETO

2

PARTES

4

ALGORITMOS

13

TESTES

9+

VISUALIZAÇÕES

FLUXO DO SISTEMA

CSV

dados brutos

Graph

lista de adj.

Metrics

graus, densidade

Algorithms

BFS/DFS/Dijkstra/BF

JSON / CSV

saídas estruturadas

HTML

visualização

PARTES DO PROJETO